

Registro LLM

Este archivo contiene una descripción de las partes del desarrollo de la aplicación se usó algún LLM. No contendrá el prompt de cada caso de uso ni la respuesta completa que dió la LLM, pero sí una descripción de porqué fue necesario y un resumen de la respuesta que dió la IA.

Para empezar, al crear el proyecto, le pedimos consejos a la IA para poder crear la estructura del proyecto y qué dependencias usar. Nos sugirió usar **psycopg2** para trabajar con PostgreSQL, esto porque ya habíamos decidido usar **Flask** para correr nuestro backend.

A la hora de conectar con PostgreSQL le preguntamos al agente cómo podríamos conectar el archivo server.py con la base de datos, a lo que nos explicó cómo configurar el setup, el cual pedía el **host, puerto, nombre, usuario y contraseña** de la base de datos. También nos dió una breve explicación de cómo evitar las **SQL Injection**.

Ante una sugerencia de la IA, le preguntamos cómo podríamos crear una **connection pool** (con el objetivo de evitar crear una nueva conexión cada que quisiéramos hacer una consulta a la base de datos). Nos enseñó cómo crearla y nos ayudó a hacer funciones que permitieran obtener un acceso a la base de datos con el pool y, luego de usarlo, devolverlo al pool.

Mientras creamos el proyecto, tuvimos la idea de crear una tabla **Jugador_en_Equipo**, que relacionaría un jugador con un equipo y tuviera como llave primaria la id del equipo y el gamertag del jugador. Eventualmente nos dimos cuenta de qué esto traería problemas y decidimos que la tabla Jugador tuviera una referencia foránea a la tabla de Equipo. Mientras sucedía este cambio, nos encontramos con un debate, el hecho de que la tabla **Equipo** tenía como llave foránea la tabla **Jugador** y viceversa, esto generaba el famoso **Problema Circular**. Nuestra solución, sugerida por la IA, fue usar **Deferrable Constraints**, lo que le dice a la base de datos “No revises esta regla ahora mismo; espera a que yo termine toda mi transacción (**COMMIT**) para validar”, solucionando así nuestro problema circular.

Es importante mencionar también que en muchos momentos, a lo largo del desarrollo de la aplicación, tuvimos problemas con consultas y requerimos de la ayuda de la IA, resolviendo nuestras dudas, errores y retroalimentando nuestros conocimientos. Algo que mencionar y que ocurrió muchas veces en el desarrollo del proyecto fue la confusión de cómo era el orden de SQL, a lo que la LLM nos explicó que el orden en el

que se ejecutaba una query era **FROM & JOIN-WHERE-GROUP BY-HAVING-SELECT**, y que si se rompía este orden, la consulta tiraría error.

Mientras trabajabamos en el **ranking**, nos dimos cuenta que debíamos dividir las etapas Fase de Grupos, Semifinal y Final en Fase de Grupos y Eliminatorias, para poder organizar mejor el ranking. También, cuando lo necesitamos, pedimos ayuda y retroalimentación en cómo usar las **WindowFunction**.

Finalmente, a la hora de crear el **frontend**, le preguntamos al agente cómo podríamos crearlo y conectarlo con nuestro **backend**, a lo que nos sugirió usar **Jinja2**, que viene incluido con Flask. Nos comentó cómo usar Flask para renderizar las páginas html, lo que hacíamos con la ayuda de la función **render_template()**, que recibe como argumento el nombre del archivo html. También nos explicó cómo configurar la aplicación para conectar los directorios del frontend con el backend. Además de cómo crear un archivo base.html que contiene todo lo común entre las páginas y un bloque donde cada página inyecta su contenido html.